

# Ausführliche Dokumentation: Countdown Timer (Java Swing)

Benjamin Malicke ©

18. Mai 2026

## Inhaltsverzeichnis

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| <b>1</b> | <b>Einleitung</b>                              | <b>5</b>  |
| 1.1      | Ziele des Programms                            | 5         |
| 1.2      | Architektur des Programms                      | 5         |
| 1.3      | Klasse CountdownWindowBuilder                  | 5         |
| 1.3.1    | Aufgabe der Klasse                             | 5         |
| 1.4      | Kompletter Quellcode                           | 6         |
| 1.5      | Detaillierte Erklärung der Variablen           | 7         |
| 1.6      | Timer-Logik                                    | 7         |
| 1.7      | Blinkeffekt                                    | 7         |
| 1.8      | Farblogik                                      | 8         |
| 1.9      | Popup für Zeiteingabe                          | 8         |
| 1.10     | Klasse RoundButton                             | 8         |
| 1.10.1   | Aufgabe                                        | 8         |
| 1.11     | Kompletter Code                                | 8         |
| 1.12     | Erklärung                                      | 8         |
| 1.13     | Klasse TimeInputDialog                         | 8         |
| 1.13.1   | Aufgabe                                        | 8         |
| 1.14     | Kompletter Code                                | 9         |
| 1.15     | Erklärung                                      | 9         |
| 1.16     | Schnell-Fazit                                  | 9         |
| <b>2</b> | <b>Importe und Variablendeklarationen</b>      | <b>9</b>  |
| 2.1      | Importe                                        | 9         |
| 2.2      | Klassenkopf                                    | 10        |
| 2.3      | GUI-Komponenten (Instanzvariablen)             | 10        |
| 2.4      | Timer-Logik                                    | 11        |
| 2.5      | Zustandsvariablen                              | 11        |
| 2.6      | Menü-Komponenten                               | 11        |
| 2.7      | Farben (Designsystem)                          | 12        |
| <b>3</b> | <b>Konstruktor der Hauptklasse</b>             | <b>12</b> |
| 3.1      | Vollständiger Konstruktor-Code                 | 12        |
| 3.2      | Schrittweise Erklärung des Konstruktorablaufs  | 14        |
| 3.2.1    | 1. Fensterkonfiguration                        | 14        |
| 3.2.2    | 2. ContentPane (Hauptlayout)                   | 14        |
| 3.2.3    | 3. Zentrale Anzeige (Titel, Zeit, Progressbar) | 14        |
| 3.2.4    | 4. Button-Bereich                              | 15        |
| 3.2.5    | 5. Haupt-Timer                                 | 15        |
| 3.2.6    | 6. Puls-Timer (Blinkeffekt)                    | 15        |
| 3.2.7    | 7. Event-Handling (Benutzerinteraktion)        | 15        |

|          |                                                                                                     |           |
|----------|-----------------------------------------------------------------------------------------------------|-----------|
| <b>4</b> | <b>Methode <code>startPause()</code></b>                                                            | <b>15</b> |
| 4.1      | Vollständiger Quellcode                                                                             | 16        |
| 4.2      | Allgemeine Funktion der Methode                                                                     | 16        |
| 4.3      | Fall 1: Timer ist gestoppt (Start-Zustand)                                                          | 16        |
| 4.3.1    | Abbruchbedingung                                                                                    | 16        |
| 4.3.2    | Start der Timer                                                                                     | 16        |
| 4.3.3    | Statusänderung                                                                                      | 17        |
| 4.4      | Fall 2: Timer läuft (Pause-Zustand)                                                                 | 17        |
| 4.4.1    | Timer stoppen                                                                                       | 17        |
| 4.4.2    | Statuswechsel                                                                                       | 17        |
| 4.4.3    | Anzeige stabilisieren                                                                               | 17        |
| 4.5      | Zusammenfassung                                                                                     | 18        |
| <b>5</b> | <b>Methode <code>reset()</code></b>                                                                 | <b>18</b> |
| 5.1      | Vollständiger Quellcode                                                                             | 18        |
| 5.2      | Allgemeine Funktion der Methode                                                                     | 18        |
| 5.3      | Schritt 1: Stoppen der Timer                                                                        | 18        |
| 5.4      | Schritt 2: Zurücksetzen des Zustands                                                                | 19        |
| 5.5      | Schritt 3: Aktualisierung der Anzeige                                                               | 19        |
| 5.6      | Schritt 4: UI-Zustand zurücksetzen                                                                  | 19        |
| 5.7      | Schritt 5: Button-Zustand zurücksetzen                                                              | 19        |
| 5.8      | Zusammenfassung                                                                                     | 19        |
| <b>6</b> | <b>Methode <code>openDialog()</code></b>                                                            | <b>20</b> |
| 6.1      | Vollständiger Quellcode                                                                             | 20        |
| 6.2      | Allgemeine Funktion der Methode                                                                     | 21        |
| 6.3      | Schritt 1: Erstellen des Dialogfensters                                                             | 21        |
| 6.4      | Schritt 2: Eingabefelder                                                                            | 21        |
| 6.5      | Schritt 3: OK-Button und Eingabvalidierung                                                          | 21        |
| 6.6      | Schritt 4: Validierung der Eingaben                                                                 | 22        |
| 6.7      | Schritt 5: Übernahme der neuen Zeit                                                                 | 22        |
| 6.8      | Schritt 6: Abschluss                                                                                | 22        |
| 6.9      | Fehlerbehandlung                                                                                    | 22        |
| 6.10     | Zusammenfassung                                                                                     | 22        |
| <b>7</b> | <b>Hilfsmethoden: <code>setTime()</code>, <code>updateDisplay()</code> und <code>style()</code></b> | <b>23</b> |
| 7.1      | Methode <code>setTime()</code>                                                                      | 23        |
| 7.1.1    | Vollständiger Quellcode                                                                             | 23        |
| 7.1.2    | Erklärung                                                                                           | 23        |
| 7.2      | Methode <code>updateDisplay()</code>                                                                | 23        |
| 7.2.1    | Vollständiger Quellcode                                                                             | 23        |
| 7.2.2    | Erklärung der Zeitformatierung                                                                      | 24        |
| 7.2.3    | Farblogik der Anzeige                                                                               | 24        |
| 7.2.4    | Fortschrittsberechnung                                                                              | 24        |
| 7.3      | Methode <code>style()</code>                                                                        | 24        |
| 7.3.1    | Vollständiger Quellcode                                                                             | 24        |
| 7.3.2    | Erklärung                                                                                           | 25        |
| 7.4      | Zusammenfassung                                                                                     | 25        |
| <b>8</b> | <b>Methode <code>createMenu()</code></b>                                                            | <b>25</b> |
| 8.1      | Vollständiger Quellcode                                                                             | 25        |
| 8.2      | Aufbau der Menüleiste                                                                               | 26        |
| 8.3      | Menüeinträge und ihre Funktion                                                                      | 26        |
| 8.4      | Tastenkombinationen (Keyboard Shortcuts)                                                            | 26        |

|           |                                                              |           |
|-----------|--------------------------------------------------------------|-----------|
| 8.5       | Zuweisung der Shortcuts . . . . .                            | 27        |
| 8.6       | Bedeutung für die Benutzerfreundlichkeit . . . . .           | 27        |
| 8.7       | Zusammenfassung . . . . .                                    | 27        |
| <b>9</b>  | <b>Klasse RoundButton</b>                                    | <b>27</b> |
| 9.1       | Vollständiger Quellcode . . . . .                            | 27        |
| 9.2       | Grundidee der Klasse . . . . .                               | 28        |
| 9.3       | Konstruktor und grundlegendes Design . . . . .               | 28        |
| 9.4       | Mausverhalten . . . . .                                      | 29        |
| 9.5       | Individuelles Rendering . . . . .                            | 29        |
| 9.6       | Klickbereich (Shape-Logik) . . . . .                         | 29        |
| 9.7       | Bedeutung für das Design . . . . .                           | 30        |
| 9.8       | Zusammenfassung . . . . .                                    | 30        |
| <b>10</b> | <b>Klasse TimeInputDialog</b>                                | <b>30</b> |
| 10.1      | Vollständiger Quellcode . . . . .                            | 30        |
| 10.2      | Grundidee der Klasse . . . . .                               | 31        |
| 10.3      | Verwendung von inneren Klassen (Swing-Komponenten) . . . . . | 32        |
| 10.4      | Lambda-Ausdrücke für Event Handling . . . . .                | 32        |
| 10.4.1    | Erklärung . . . . .                                          | 32        |
| 10.4.2    | Vergleich zu klassischer Schreibweise . . . . .              | 32        |
| 10.5      | Modale Fensterlogik . . . . .                                | 32        |
| 10.6      | Datenzugriff über Getter-Methoden . . . . .                  | 33        |
| 10.7      | Bedeutung für die Softwarearchitektur . . . . .              | 33        |
| 10.8      | Zusammenfassung . . . . .                                    | 33        |
| <b>11</b> | <b>Startpunkt der Anwendung: main()-Methode</b>              | <b>33</b> |
| 11.1      | Vollständiger Quellcode . . . . .                            | 33        |
| 11.2      | Aufgabe der main-Methode . . . . .                           | 34        |
| 11.3      | Swing Event Dispatch Thread (EDT) . . . . .                  | 34        |
| 11.4      | Verwendung eines Lambda-Ausdrucks . . . . .                  | 34        |
| 11.5      | Erzeugung der GUI . . . . .                                  | 34        |
| 11.6      | Trennung von Logik und Darstellung . . . . .                 | 35        |
| 11.7      | Zusammenfassung . . . . .                                    | 35        |
| <b>12</b> | <b>Mögliche Erweiterungen der Anwendung</b>                  | <b>35</b> |
| 12.1      | Erweiterung: Mehrere Timer gleichzeitig . . . . .            | 35        |
| 12.2      | Erweiterung: Speicherung von Einstellungen . . . . .         | 35        |
| 12.3      | Erweiterung: Alarmfunktionen . . . . .                       | 36        |
| 12.4      | Erweiterung: Benutzerdefinierte Oberfläche . . . . .         | 36        |
| 12.5      | Erweiterung: Verbesserung der Zeiteingabe . . . . .          | 36        |
| 12.6      | Erweiterung: Mobile oder Web-Version . . . . .               | 36        |
| 12.7      | Zusammenfassung . . . . .                                    | 36        |
| <b>13</b> | <b>Fazit und Zusammenfassung</b>                             | <b>36</b> |
| 13.1      | Erreichte Ziele . . . . .                                    | 37        |
| 13.2      | Technische Umsetzung . . . . .                               | 37        |
| 13.3      | Stärken der Anwendung . . . . .                              | 37        |
| 13.4      | Herausforderungen während der Entwicklung . . . . .          | 37        |
| 13.5      | Ausblick . . . . .                                           | 38        |
| 13.6      | Schlusswort . . . . .                                        | 38        |
| <b>A</b>  | <b>Klasse CountdownWindowBuilder</b>                         | <b>39</b> |
| <b>B</b>  | <b>Klasse RoundButton</b>                                    | <b>44</b> |



# 1 Einleitung

Diese Software ist ein grafischer Countdown-Timer, der in Java mit Swing entwickelt wurde. Die Anwendung dient dazu, Zeit herunterzuzählen und dem Benutzer visuelles sowie akustisches Feedback zu geben.

## 1.1 Ziele des Programms

Das Programm wurde entwickelt, um folgende Funktionen zu erfüllen:

- Eingabe einer Zeit (Minuten und Sekunden)
- Starten, Pausieren und Zurücksetzen des Timers
- visuelle Darstellung der verbleibenden Zeit
- Farbwechsel abhängig von Restzeit
- Blinkereffekt in den letzten Sekunden
- akustisches Signal am Ende
- Menüführung mit Tastenkürzeln

## 1.2 Architektur des Programms

Die Anwendung besteht aus drei Klassen:

- CountdownWindowBuilder (Hauptfenster)
- RoundButton (benutzerdefinierte UI-Komponente)
- TimeInputDialog (Eingabedialog für Zeit)

Diese Struktur trennt:

- Logik (Timer)
- Benutzeroberfläche
- Eingabedialoge

## 1.3 Klasse CountdownWindowBuilder

Diese Klasse ist das zentrale Element der Anwendung.

### 1.3.1 Aufgabe der Klasse

Die Klasse übernimmt:

- Aufbau des Hauptfensters
- Steuerung der Timer-Logik
- Verwaltung der UI-Komponenten
- Verarbeitung von Benutzeraktionen

## 1.4 Kompletter Quellcode

```
1 package Timer;
2
3 import javax.swing.*;
4 import javax.swing.border.EmptyBorder;
5 import java.awt.*;
6 import java.awt.event.KeyEvent;
7
8 public class CountdownWindowBuilder extends JFrame {
9
10     private JPanel contentPane;
11     private JLabel titleLabel;
12     private JLabel timeLabel;
13
14     private JButton startPauseButton;
15     private JButton resetButton;
16     private JButton newButton;
17
18     private JProgressBar progressBar;
19
20     private Timer timer;
21     private Timer pulseTimer;
22
23     private int initialSeconds = 0;
24     private int remainingSeconds = 0;
25     private boolean running = false;
26
27     private JMenuItem newItem, startPauseItem, resetItem, exitItem,
28         aboutItem;
29
30     private final Color bg = new Color(24, 24, 28);
31     private final Color panelBg = new Color(32, 32, 38);
32     private final Color text = new Color(235, 235, 235);
33     private final Color accent = new Color(70, 130, 255);
34     private final Color green = new Color(46, 125, 50);
35     private final Color red = new Color(198, 40, 40);
36
37     public static void main(String[] args) {
38         SwingUtilities.invokeLater(() -> {
39             CountdownWindowBuilder f = new CountdownWindowBuilder();
40             f.setVisible(true);
41         });
42     }
43
44     public CountdownWindowBuilder() {
45
46         setTitle("Countdown");
47         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
48         setSize(700, 500);
49         setLocationRelativeTo(null);
50         setResizable(false);
51
52         contentPane = new JPanel();
53         contentPane.setBorder(new EmptyBorder(20, 20, 20, 20));
54         contentPane.setBackground(bg);
55         contentPane.setLayout(new BorderLayout(15, 15));
56         setContentPane(contentPane);
57
58         createMenu();
```

```

59     JPanel center = new JPanel(new BorderLayout(10, 10));
60     center.setOpaque(false);
61
62     titleLabel = new JLabel("Countdown", SwingConstants.CENTER);
63     timeLabel = new JLabel("00:00", SwingConstants.CENTER);
64
65     progressBar = new JProgressBar(0, 100);
66
67     center.add(titleLabel, BorderLayout.NORTH);
68     center.add(timeLabel, BorderLayout.CENTER);
69     center.add(progressBar, BorderLayout.SOUTH);
70
71     contentPane.add(center, BorderLayout.CENTER);
72 }
73 }

```

## 1.5 Detaillierte Erklärung der Variablen

- initialSeconds: ursprünglich eingestellte Zeit
- remainingSeconds: aktuell verbleibende Zeit
- running: Status ob Timer läuft
- timer: Haupt-Timer (1 Sekunde)
- pulseTimer: Blink-Timer (300 ms)

## 1.6 Timer-Logik

Der Haupt-Timer reduziert jede Sekunde die Zeit:

```

1 timer = new Timer(1000, e -> {
2     remainingSeconds--;
3     updateDisplay();
4
5     if (remainingSeconds <= 0) {
6         timer.stop();
7         pulseTimer.stop();
8         Toolkit.getDefaultToolkit().beep();
9     }
10 });

```

### Erklärung:

- jede Sekunde wird heruntergezählt
- Anzeige wird aktualisiert
- bei 0 wird gestoppt und Signalton ausgegeben

## 1.7 Blinkeffekt

```

1 pulseTimer = new Timer(300, e -> {
2     if (remainingSeconds <= 5 && remainingSeconds > 0) {
3         timeLabel.setVisible(!timeLabel.isVisible());
4     } else {
5         timeLabel.setVisible(true);
6     }
7 });

```

**Erklärung:** In den letzten 5 Sekunden blinkt die Anzeige durch Ein-/Ausblenden.

## 1.8 Farblogik

- > 10 Sekunden: blau
- 10 Sekunden: rot
- 5 Sekunden: stärker rot + Blinkeffekt

## 1.9 Popup für Zeiteingabe

Der Benutzer kann eine neue Zeit eingeben:

```
1 JDialog d = new JDialog(this, "Neue Zeit", true);
```

**Validierung:**

- Minuten  $\geq 0$
- Sekunden zwischen 0 und 59
- nur Zahlen erlaubt

## 1.10 Klasse RoundButton

### 1.10.1 Aufgabe

Diese Klasse erstellt runde Buttons mit Hover-Effekt.

## 1.11 Kompletter Code

```
1 package Timer;
2
3 import javax.swing.*;
4 import java.awt.*;
5 import java.awt.event.MouseAdapter;
6 import java.awt.event.MouseEvent;
7 import java.awt.geom.Ellipse2D;
8
9 public class RoundButton extends JButton {
10
11     private Shape shape;
12
13     public RoundButton(String text) {
14         super(text);
15         setContentAreaFilled(false);
16         setFocusPainted(false);
17         setBorderPainted(false);
18         setOpaque(false);
19     }
20 }
```

## 1.12 Erklärung

Die Klasse überschreibt das Zeichungsverhalten des Buttons und erstellt eine kreisförmige Darstellung.

## 1.13 Klasse TimeInputDialog

### 1.13.1 Aufgabe

Diese Klasse stellt einen separaten Dialog zur Zeiteingabe bereit.



## 1.14 Kompletter Code

```
1 package Timer;
2
3 import javax.swing.*;
4 import java.awt.*;
5
6 public class TimeInputDialog extends JDialog {
7
8     private JTextField minutesField;
9     private JTextField secondsField;
10
11     public TimeInputDialog(JFrame parent) {
12         super(parent, "Neue Zeit", true);
13     }
14 }
```

## 1.15 Erklärung

Der Dialog:

- nimmt Min/Sek Eingaben entgegen
- validiert Benutzerinput
- liefert Werte zurück an Hauptprogramm

## 1.16 Schnell-Fazit

Die Anwendung zeigt:

- objektorientierte Programmierung
- GUI-Design mit Swing
- Ereignisgesteuerte Programmierung
- Timer- und UI-Synchronisation

Das Projekt ist modular aufgebaut und leicht erweiterbar.

# 2 Importe und Variablendeklarationen

## 2.1 Importe

```
1 package Timer;
2
3 import javax.swing.*;
4 import javax.swing.border.EmptyBorder;
5 import java.awt.*;
6 import java.awt.event.KeyEvent;
```

## Erklärung der Imports

Die verwendeten Bibliotheken stammen hauptsächlich aus der Java Swing- und AWT-Bibliothek:

- `javax.swing.*`  
Enthält alle wichtigen GUI-Komponenten wie Fenster (`JFrame`), Buttons, Labels und Dialoge.
- `javax.swing.border.EmptyBorder`  
Wird verwendet, um Abstände (Padding) innerhalb von Panels zu definieren.
- `java.awt.*`  
Enthält grundlegende grafische Klassen wie Farben (`Color`), Layouts und Zeichenelemente.
- `java.awt.event.KeyEvent`  
Wird für Tastatursteuerung und Tastenkombinationen im Menü verwendet.

## 2.2 Klassenkopf

```
1 public class CountdownWindowBuilder extends JFrame {
```

Die Klasse `CountdownWindowBuilder` erweitert `JFrame`. Dadurch wird sie zu einem vollwertigen Fenster mit GUI-Funktionalität.

### Bedeutung

- `extends JFrame` bedeutet: Die Klasse ist ein Fenster
- Dadurch können Swing-Komponenten direkt eingebettet werden
- Die Klasse bildet das Hauptfenster der Anwendung

## 2.3 GUI-Komponenten (Instanzvariablen)

```
1 private JPanel contentPane;  
2 private JLabel titleLabel;  
3 private JLabel timeLabel;  
4  
5 private JButton startPauseButton;  
6 private JButton resetButton;  
7 private JButton newButton;  
8  
9 private JProgressBar progressBar;
```

### Erklärung der GUI-Komponenten

- `contentPane`  
Das Haupt-Panel des Fensters, in dem alle Elemente platziert werden.
- `titleLabel`  
Zeigt den Titel der Anwendung an (z.B. "Countdown").
- `timeLabel`  
Zeigt die aktuell verbleibende Zeit im Format MM:SS an.
- `startPauseButton`  
Startet oder pausiert den Countdown.

- `resetButton`  
Setzt den Timer auf die Ausgangszeit zurück.
- `newButton`  
Öffnet den Dialog zur Eingabe einer neuen Zeit.
- `progressBar`  
Visualisiert den Fortschritt des Countdowns als Prozentbalken.

## 2.4 Timer-Logik

```
1 private Timer timer;
2 private Timer pulseTimer;
```

### Erklärung

- `timer`  
Haupttimer, der jede Sekunde den Countdown reduziert.
- `pulseTimer`  
Zusätzlicher Timer, der in kurzen Intervallen (300 ms) den Blinkereffekt in den letzten Sekunden erzeugt.

## 2.5 Zustandsvariablen

```
1 private int initialSeconds = 0;
2 private int remainingSeconds = 0;
3 private boolean running = false;
```

### Erklärung

- `initialSeconds`  
Speichert die ursprünglich eingestellte Gesamtzeit.
- `remainingSeconds`  
Enthält die aktuell verbleibende Zeit während des Countdowns.
- `running`  
Gibt an, ob der Timer aktuell läuft oder pausiert ist.

## 2.6 Menü-Komponenten

```
1 private JMenuItem newItem, startPauseItem, resetItem, exitItem,
   aboutItem;
```

### Erklärung

Diese Menüeinträge ermöglichen die Steuerung der Anwendung über ein klassisches Menüsystem:

- `newItem` – neue Zeit eingeben
- `startPauseItem` – Start/Pause Funktion
- `resetItem` – Zurücksetzen des Timers
- `exitItem` – Programm beenden
- `aboutItem` – Informationen über die Anwendung

## 2.7 Farben (Designsystem)

```
1 private final Color bg = new Color(24, 24, 28);
2 private final Color panelBg = new Color(32, 32, 38);
3 private final Color text = new Color(235, 235, 235);
4 private final Color accent = new Color(70, 130, 255);
5 private final Color green = new Color(46, 125, 50);
6 private final Color red = new Color(198, 40, 40);
```

### Erklärung

Diese Farbdefinitionen bilden das Designsystem der Anwendung:

- **bg** – Hintergrundfarbe des Fensters
- **panelBg** – Hintergrundfarbe für Panels
- **text** – Standard Textfarbe
- **accent** – Hervorhebungsfarbe (Blau)
- **green** – Start/Bestätigen Buttons
- **red** – Reset/Stop Buttons

## 3 Konstruktor der Hauptklasse

Der Konstruktor `CountdownWindowBuilder()` ist der zentrale Aufbaupunkt der gesamten Anwendung. Hier wird das Hauptfenster initialisiert, die Benutzeroberfläche erstellt und die Timer-Logik vorbereitet.

### 3.1 Vollständiger Konstruktor-Code

```
1 public CountdownWindowBuilder() {
2
3     setTitle("Countdown");
4     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
5     setSize(700, 500);
6     setLocationRelativeTo(null);
7     setResizable(false);
8
9     contentPane = new JPanel();
10    contentPane.setBorder(new EmptyBorder(20, 20, 20, 20));
11    contentPane.setBackground(bg);
12    contentPane.setLayout(new BorderLayout(15, 15));
13    setContentPane(contentPane);
14
15    createMenu();
16
17    JPanel center = new JPanel(new BorderLayout(10, 10));
18    center.setOpaque(false);
19
20    titleLabel = new JLabel("Countdown", SwingConstants.CENTER);
21    titleLabel.setFont(new Font("SansSerif", Font.BOLD, 30));
22    titleLabel.setForeground(text);
23
24    timeLabel = new JLabel("00:00", SwingConstants.CENTER);
25    timeLabel.setFont(new Font("SansSerif", Font.BOLD, 64));
```

```

26     timeLabel.setForeground(accent);
27
28     progressBar = new JProgressBar(0, 100);
29     progressBar.setStringPainted(true);
30     progressBar.setValue(100);
31     progressBar.setString("100%");
32     progressBar.setForeground(accent);
33     progressBar.setBackground(panelBg);
34
35     center.add(titleLabel, BorderLayout.NORTH);
36     center.add(timeLabel, BorderLayout.CENTER);
37     center.add(progressBar, BorderLayout.SOUTH);
38
39     contentPane.add(center, BorderLayout.CENTER);
40
41     JPanel buttons = new JPanel(new FlowLayout(FlowLayout.CENTER, 15, 5)
42     );
43     buttons.setOpaque(false);
44
45     newButton = new RoundButton("Neu");
46     startPauseButton = new RoundButton("Start");
47     resetButton = new RoundButton("Reset");
48
49     style(newButton, accent);
50     style(startPauseButton, green);
51     style(resetButton, red);
52
53     resetButton.setEnabled(false);
54
55     buttons.add(newButton);
56     buttons.add(startPauseButton);
57     buttons.add(resetButton);
58
59     contentPane.add(buttons, BorderLayout.SOUTH);
60
61     timer = new Timer(1000, e -> {
62
63         remainingSeconds--;
64         updateDisplay();
65
66         if (remainingSeconds <= 0) {
67             timer.stop();
68             pulseTimer.stop();
69
70             running = false;
71             startPauseButton.setText("Start");
72
73             timeLabel.setVisible(true);
74             timeLabel.setText("Fertig!");
75             timeLabel.setForeground(Color.RED);
76
77             Toolkit.getDefaultToolkit().beep();
78         }
79     });
80
81     pulseTimer = new Timer(300, e -> {
82
83         if (remainingSeconds <= 5 && remainingSeconds > 0) {
84             timeLabel.setVisible(!timeLabel.isVisible());
85         } else {
86             timeLabel.setVisible(true);

```

```

86     }
87   });
88
89   pulseTimer.start();
90
91   newButton.addActionListener(e -> openDialog());
92   startPauseButton.addActionListener(e -> startPause());
93   resetButton.addActionListener(e -> reset());
94
95   newItem.addActionListener(e -> openDialog());
96   startPauseItem.addActionListener(e -> startPauseButton.doClick());
97   resetItem.addActionListener(e -> reset());
98   exitItem.addActionListener(e -> System.exit(0));
99
100  aboutItem.addActionListener(e -> JOptionPane.showMessageDialog(
101      this,
102      "Countdown Timer",
103      "About",
104      JOptionPane.INFORMATION_MESSAGE
105  ));
106 }

```

## 3.2 Schrittweise Erklärung des Konstruktorablaufs

Der Konstruktor lässt sich in mehrere logische Abschnitte unterteilen:

### 3.2.1 1. Fensterkonfiguration

- `setTitle("Countdown")` legt den Fenstertitel fest.
- `setDefaultCloseOperation` beendet das Programm beim Schließen.
- `setSize(700, 500)` definiert die Fenstergröße.
- `setLocationRelativeTo(null)` zentriert das Fenster auf dem Bildschirm.
- `setResizable(false)` verhindert das Verändern der Fenstergröße.

Diese Einstellungen definieren das Grundverhalten des Hauptfensters.

### 3.2.2 2. ContentPane (Hauptlayout)

- Erstellung eines Haupt-JPanels als Container für alle UI-Elemente
- Einfügen eines inneren Abstands (Padding) mittels `EmptyBorder`
- Setzen eines dunklen Hintergrunddesigns
- Verwendung eines `BorderLayout` zur Strukturierung

Dieses Panel bildet die Basis der gesamten Benutzeroberfläche.

### 3.2.3 3. Zentrale Anzeige (Titel, Zeit, Progressbar)

Der mittlere Bereich zeigt die wichtigsten Informationen:

- **Titel** – beschreibt die Anwendung
- **Zeitlabel** – zeigt den Countdown im Format MM:SS
- **Progressbar** – visualisiert den Fortschritt prozentual

Die Komponenten werden in einem eigenen Panel mit `BorderLayout` organisiert.

### 3.2.4 4. Button-Bereich

Der untere Bereich enthält die Steuerung:

- **Neu** – öffnet Eingabedialog
- **Start** – startet den Countdown
- **Reset** – setzt den Timer zurück

Die Buttons werden mit individuellen Farben gestaltet und optisch hervorgehoben.

### 3.2.5 5. Haupt-Timer

Der Haupt-Timer arbeitet im 1-Sekunden-Takt:

- reduziert die verbleibende Zeit
- aktualisiert die Anzeige
- beendet den Countdown bei 0
- gibt ein akustisches Signal (Beep) aus

Dieser Timer ist das Herzstück der Zeitlogik.

### 3.2.6 6. Puls-Timer (Blinkeffekt)

Der zweite Timer erzeugt visuelles Feedback:

- aktiviert in den letzten 5 Sekunden
- lässt die Zeitanzeige blinken
- deaktiviert sich außerhalb dieses Bereichs

Dies erhöht die Aufmerksamkeit des Benutzers kurz vor Ablauf.

### 3.2.7 7. Event-Handling (Benutzerinteraktion)

Alle Benutzeraktionen werden über Event-Listener verarbeitet:

- Buttons steuern Start, Pause und Reset
- Menüeinträge spiegeln diese Funktionen wider
- Exit beendet das Programm
- About zeigt Programminformationen

Die Menüstruktur sorgt für zusätzliche Bedienbarkeit und Tastenkürzel-Unterstützung.

## 4 Methode `startPause()`

Die Methode `startPause()` steuert den zentralen Zustandswechsel des Timers zwischen **Start** und **Pause**. Sie ist dafür verantwortlich, den Ablauf der Anwendung dynamisch zu kontrollieren.

## 4.1 Vollständiger Quellcode

```
1 private void startPause() {
2
3     if (!running) {
4
5         if (remainingSeconds <= 0) return;
6
7         timer.start();
8         pulseTimer.start();
9
10        running = true;
11        startPauseButton.setText("Pause");
12        resetButton.setEnabled(true);
13
14    } else {
15        timer.stop();
16        pulseTimer.stop();
17
18        running = false;
19        startPauseButton.setText("Fortsetzen");
20
21        timeLabel.setVisible(true);
22    }
23 }
```

## 4.2 Allgemeine Funktion der Methode

Diese Methode entscheidet abhängig vom aktuellen Zustand (**running**), ob der Timer gestartet oder pausiert wird.

- Wenn der Timer **nicht läuft**, wird er gestartet
- Wenn der Timer **läuft**, wird er pausiert

Damit bildet die Methode die wichtigste Steuerlogik der gesamten Anwendung.

## 4.3 Fall 1: Timer ist gestoppt (Start-Zustand)

```
1 if (!running) {
```

Dieser Block wird ausgeführt, wenn der Timer aktuell nicht aktiv ist.

### 4.3.1 Abbruchbedingung

```
1 if (remainingSeconds <= 0) return;
```

**Erklärung:**

- Wenn keine gültige Zeit gesetzt ist, wird die Methode sofort beendet
- verhindert Start mit 0 Sekunden

### 4.3.2 Start der Timer

```
1 timer.start();
2 pulseTimer.start();
```

**Erklärung:**



- timer startet den eigentlichen Countdown (1 Sekunde Takt)
- pulseTimer startet den Blinkereffekt in den letzten Sekunden

### 4.3.3 Statusänderung

```
1 running = true;
2 startPauseButton.setText("Pause");
3 resetButton.setEnabled(true);
```

#### Erklärung:

- Der Status wird auf aktiv gesetzt
- Der Buttontext wird auf "Pause" geändert
- Reset wird aktiviert, da ein laufender Timer zurückgesetzt werden darf

## 4.4 Fall 2: Timer läuft (Pause-Zustand)

```
1 else {
```

Dieser Block wird ausgeführt, wenn der Timer bereits aktiv ist.

### 4.4.1 Timer stoppen

```
1 timer.stop();
2 pulseTimer.stop();
```

#### Erklärung:

- beide Timer werden gestoppt
- keine Zeit wird mehr reduziert
- kein Blinken mehr aktiv

### 4.4.2 Statuswechsel

```
1 running = false;
2 startPauseButton.setText("Fortsetzen");
```

#### Erklärung:

- der Zustand wird auf "nicht laufend" gesetzt
- der Button zeigt nun "Fortsetzen"
- der Benutzer kann den Timer später an gleicher Stelle weiterlaufen lassen

### 4.4.3 Anzeige stabilisieren

```
1 timeLabel.setVisible(true);
```

#### Erklärung:

- Falls der Blinkereffekt aktiv war, wird die Anzeige wieder stabil sichtbar gemacht
- verhindert Flackern im Pause-Zustand

## 4.5 Zusammenfassung

Die Methode `startPause()` implementiert eine klassische Zustandslogik:

- Start → aktiviert Timer und UI-Feedback
- Pause → stoppt Timer ohne Zeitverlust
- Fortsetzen → setzt exakt an der gleichen Stelle fort

Damit bildet sie das zentrale Steuerungselement der Anwendung.

## 5 Methode `reset()`

Die Methode `reset()` setzt den Timer vollständig in den Ausgangszustand zurück. Sie stellt sicher, dass sowohl die Zeit als auch die Benutzeroberfläche wieder den ursprünglichen Zustand annehmen.

### 5.1 Vollständiger Quellcode

```
1 private void reset() {  
2  
3     timer.stop();  
4     pulseTimer.stop();  
5  
6     running = false;  
7     remainingSeconds = initialSeconds;  
8  
9     updateDisplay();  
10  
11    timeLabel.setVisible(true);  
12    timeLabel.setForeground(accent);  
13  
14    startPauseButton.setText("Start");  
15    resetButton.setEnabled(false);  
16 }
```

### 5.2 Allgemeine Funktion der Methode

Die Methode stellt den Ursprungszustand des Countdowns wieder her.

Dabei werden zwei Bereiche zurückgesetzt:

- **Logikzustand** (Zeit, Timer, Status)
- **Benutzeroberfläche** (Anzeige, Farben, Buttons)

### 5.3 Schritt 1: Stoppen der Timer

```
1 timer.stop();  
2 pulseTimer.stop();
```

**Erklärung:**

- beide Timer werden sofort gestoppt
- verhindert weitere Zeitveränderung
- beendet auch den Blinkereffekt

## 5.4 Schritt 2: Zurücksetzen des Zustands

```
1 running = false;  
2 remainingSeconds = initialSeconds;
```

### Erklärung:

- der Laufstatus wird auf `false` gesetzt
- die verbleibende Zeit wird auf den Startwert zurückgesetzt
- der ursprüngliche Countdown wird wiederhergestellt

## 5.5 Schritt 3: Aktualisierung der Anzeige

```
1 updateDisplay();
```

### Erklärung:

- aktualisiert Zeitlabel und Fortschrittsbalken
- stellt sicher, dass UI und Logik synchron sind

## 5.6 Schritt 4: UI-Zustand zurücksetzen

```
1 timeLabel.setVisible(true);  
2 timeLabel.setForeground(accent);
```

### Erklärung:

- falls zuvor ein Blinkereffekt aktiv war, wird dieser beendet
- die Farbe wird auf Standardfarbe (Accent) zurückgesetzt
- sorgt für visuell stabilen Ausgangszustand

## 5.7 Schritt 5: Button-Zustand zurücksetzen

```
1 startPauseButton.setText("Start");  
2 resetButton.setEnabled(false);
```

### Erklärung:

- Start/Pause Button wird wieder auf "Start" gesetzt
- Reset-Button wird deaktiviert, da keine aktive Sitzung mehr läuft

## 5.8 Zusammenfassung

Die Methode `reset()` sorgt für einen vollständigen Neustart der Anwendung. Sie stellt sicher, dass:

- keine Timer mehr laufen
- die Zeit auf den Ausgangswert zurückgesetzt wird
- die Benutzeroberfläche wieder in den Grundzustand versetzt wird

Damit ist sie essenziell für die Benutzerkontrolle und Fehlerkorrektur während der Nutzung.

## 6 Methode openDialog()

Die Methode `openDialog()` öffnet ein modales Eingabefenster, in dem der Benutzer eine neue Zeit für den Countdown festlegen kann.

Sie verbindet Benutzerinteraktion, Eingabevalidierung und die Aktualisierung der Timer-Logik.

### 6.1 Vollständiger Quellcode

```
1 private void openDialog() {
2
3     JDialog d = new JDialog(this, "Neue Zeit", true);
4     d.setSize(280, 160);
5     d.setLocationRelativeTo(this);
6     d.setLayout(new FlowLayout());
7
8     JTextField m = new JTextField(String.valueOf(initialSeconds / 60),
9         3);
10    JTextField s = new JTextField(String.valueOf(initialSeconds % 60),
11        3);
12
13    JButton ok = new JButton("OK");
14
15    ok.addActionListener(ev -> {
16        try {
17
18            int mm = Integer.parseInt(m.getText().trim());
19            int ss = Integer.parseInt(s.getText().trim());
20
21            if (mm < 0 || ss < 0 || ss > 59) {
22                JOptionPane.showMessageDialog(d,
23                    "Bitte gueltige Zeit eingeben:\nMin >= 0\nSek
24                    0-59",
25                    "Ungueltig",
26                    JOptionPane.WARNING_MESSAGE);
27                return;
28            }
29
30            timer.stop();
31            pulseTimer.stop();
32
33            running = false;
34            startPauseButton.setText("Start");
35
36            setTime(mm, ss);
37
38            resetButton.setEnabled(true);
39
40            d.dispose();
41
42        } catch (NumberFormatException ex) {
43            JOptionPane.showMessageDialog(d,
44                "Bitte nur Zahlen eingeben!",
45                "Fehler",
46                JOptionPane.ERROR_MESSAGE);
47        }
48    });
49
50    d.add(new JLabel("Min:"));
51    d.add(m);
```

```

49     d.add(new JLabel("Sek:"));
50     d.add(s);
51     d.add(ok);
52
53     d.setVisible(true);
54 }

```

## 6.2 Allgemeine Funktion der Methode

Diese Methode öffnet ein modales Dialogfenster, das die Hauptanwendung blockiert, bis der Benutzer eine Eingabe bestätigt oder abbricht.

Die Hauptaufgaben sind:

- Eingabe von Minuten und Sekunden
- Validierung der Eingabewerte
- Übernahme der neuen Zeit in den Timer
- Rücksetzen laufender Prozesse

## 6.3 Schritt 1: Erstellen des Dialogfensters

```

1 JDialog d = new JDialog(this, "Neue Zeit", true);

```

**Erklärung:**

- erstellt ein modales Fenster (`true`)
- blockiert das Hauptfenster solange der Dialog offen ist
- Titel: "Neue Zeit"

## 6.4 Schritt 2: Eingabefelder

```

1 JTextField m = new JTextField(String.valueOf(initialSeconds / 60), 3);
2 JTextField s = new JTextField(String.valueOf(initialSeconds % 60), 3);

```

**Erklärung:**

- Minutenfeld wird automatisch aus der aktuellen Zeit berechnet
- Sekundenfeld ebenfalls aus Restwert
- erleichtert Benutzer die Anpassung bestehender Zeiten

## 6.5 Schritt 3: OK-Button und Eingabvalidierung

```

1 int mm = Integer.parseInt(m.getText().trim());
2 int ss = Integer.parseInt(s.getText().trim());

```

**Erklärung:**

- Eingaben werden als Text gelesen
- Umwandlung in Integer
- mögliche Fehler werden durch Exception abgefangen

## 6.6 Schritt 4: Validierung der Eingaben

```
1 if (mm < 0 || ss < 0 || ss > 59)
```

**Regeln:**

- Minuten müssen  $\geq 0$  sein
- Sekunden müssen zwischen 0 und 59 liegen

Bei falscher Eingabe wird eine Warnung angezeigt.

## 6.7 Schritt 5: Übernahme der neuen Zeit

- Timer wird gestoppt
- Blinkeffekt wird beendet
- Zustand wird zurückgesetzt
- neue Zeit wird gesetzt

```
1 setTime(mm, ss);
```

Diese Methode aktualisiert die interne Zeitbasis des Countdowns.

## 6.8 Schritt 6: Abschluss

- Reset-Button wird aktiviert
- Dialog wird geschlossen
- Benutzer kehrt zur Hauptanwendung zurück

```
1 d.dispose();
```

## 6.9 Fehlerbehandlung

Falls keine gültigen Zahlen eingegeben werden:

- wird eine `NumberFormatException` ausgelöst
- eine Fehlermeldung wird angezeigt
- Dialog bleibt offen

## 6.10 Zusammenfassung

Die Methode `openDialog()` ist ein zentrales Element der Benutzerinteraktion.

Sie kombiniert:

- GUI-Dialoge
- Eingabevalidierung
- Fehlerbehandlung
- Zustandsmanagement des Timers

Damit ermöglicht sie eine sichere und flexible Zeiteingabe während der Laufzeit.

## 7 Hilfsmethoden: setTime(), updateDisplay() und style()

Diese drei Methoden übernehmen zentrale Unterstützungsfunktionen der Anwendung:

- setTime() – setzt die neue Zeitbasis
- updateDisplay() – aktualisiert die Anzeige und Logik der UI
- style() – vereinheitlicht das Design der Buttons

Sie sorgen gemeinsam für eine klare Trennung zwischen Logik, Darstellung und Design.

### 7.1 Methode setTime()

#### 7.1.1 Vollständiger Quellcode

```
1 private void setTime(int m, int s) {
2
3     initialSeconds = m * 60 + s;
4     remainingSeconds = initialSeconds;
5
6     updateDisplay();
7 }
```

#### 7.1.2 Erklärung

Die Methode wandelt die eingegebene Zeit in Sekunden um und setzt sie als neue Grundlage für den Countdown.

- `m * 60 + s` konvertiert Minuten und Sekunden in Gesamtsekunden
- `initialSeconds` speichert den Ausgangswert
- `remainingSeconds` wird auf denselben Wert gesetzt
- anschließend wird die Anzeige aktualisiert

Diese Methode stellt sicher, dass jede neue Zeiteingabe korrekt in das System übernommen wird.

### 7.2 Methode updateDisplay()

#### 7.2.1 Vollständiger Quellcode

```
1 private void updateDisplay() {
2
3     int min = remainingSeconds / 60;
4     int sec = remainingSeconds % 60;
5
6     timeLabel.setText(String.format("%02d:%02d", min, sec));
7
8     // Farbe
9     if (remainingSeconds <= 5 && remainingSeconds > 0) {
10         timeLabel.setForeground(Color.RED);
11     } else if (remainingSeconds <= 10) {
12         timeLabel.setForeground(Color.RED.darker());
13     } else {
14         timeLabel.setForeground(accent);
15     }
```

```

16
17     if (initialSeconds > 0) {
18         int percent = (int) (remainingSeconds * 100.0 / initialSeconds);
19         progressBar.setValue(percent);
20         progressBar.setString(percent + "%");
21     }
22 }

```

### 7.2.2 Erklärung der Zeitformatierung

- Die Sekunden werden in Minuten und Sekunden umgerechnet
- `String.format("%02d:%02d")` sorgt für führende Nullen
- Beispiel: 5 Sekunden → 00:05

### 7.2.3 Farblogik der Anzeige

Die Farbe der Zeitanzeige verändert sich dynamisch abhängig von der verbleibenden Zeit:

- **> 10 Sekunden** → Standardfarbe (Accent, blau)
- **<= 10 Sekunden** → dunkleres Rot (Warnstufe)
- **<= 5 Sekunden** → kräftiges Rot (kritische Phase)

Diese visuelle Abstufung erhöht die Benutzeraufmerksamkeit.

### 7.2.4 Fortschrittsberechnung

- Der Fortschritt wird prozentual berechnet
- Formel: `remainingSeconds / initialSeconds`
- Umrechnung in Prozentwerte für die ProgressBar

Beispiel:

- Start: 100%
- Halbzeit: 50%
- Ende: 0%

Die ProgressBar visualisiert somit den aktuellen Fortschritt des Countdowns.

## 7.3 Methode `style()`

### 7.3.1 Vollständiger Quellcode

```

1 private void style(RoundButton b, Color c) {
2     b.setBackground(c);
3     b.setForeground(Color.WHITE);
4     b.setPreferredSize(new Dimension(90, 90));
5 }

```



### 7.3.2 Erklärung

Diese Methode dient der einheitlichen Gestaltung der Buttons.

- `setBackground(c)` setzt die individuelle Buttonfarbe
- `setForeground(Color.WHITE)` sorgt für gute Lesbarkeit
- `setPreferredSize(90x90)` sorgt für gleichmäßige Form

### 7.4 Zusammenfassung

Diese drei Methoden übernehmen zentrale Aufgaben:

- `setTime()` – Datenlogik (Zeit setzen)
- `updateDisplay()` – visuelle und logische Aktualisierung
- `style()` – Designvereinheitlichung der UI

Sie bilden gemeinsam die Verbindung zwischen Benutzerinteraktion, Logik und Darstellung.

## 8 Methode `createMenu()`

Die Methode `createMenu()` erstellt die Menüleiste der Anwendung und bindet alle wichtigen Programmfunktionen über Menüpunkte sowie Tastenkürzel ein.

Sie verbessert die Benutzerfreundlichkeit, da alle Funktionen zusätzlich zur Button-Steuerung auch über das Menü erreichbar sind.

### 8.1 Vollständiger Quellcode

```
1 private void createMenu() {
2
3     JMenuBar bar = new JMenuBar();
4     setJMenuBar(bar);
5
6     JMenu m1 = new JMenu("Menue");
7     JMenu m2 = new JMenu("About");
8
9     bar.add(m1);
10    bar.add(m2);
11
12    newItem = new JMenuItem("Neu");
13    startPauseItem = new JMenuItem("Start");
14    resetItem = new JMenuItem("Reset");
15    exitItem = new JMenuItem("Exit");
16    aboutItem = new JMenuItem("Info");
17
18    m1.add(newItem);
19    m1.add(startPauseItem);
20    m1.add(resetItem);
21    m1.add(exitItem);
22
23    m2.add(aboutItem);
24
25    int ctrl = Toolkit.getDefaultToolkit().getMenuShortcutKeyMaskEx();
26
27    newItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_N, ctrl));
28    startPauseItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_S,
        ctrl));
```

```

29     resetItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_R, ctrl)
        );
30     exitItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_X, ctrl))
        ;
31 }

```

## 8.2 Aufbau der Menüleiste

- JMenuBar ist die oberste Menüleiste des Fensters
- Sie wird mit `setJMenuBar()` dem Fenster zugewiesen
- Zwei Hauptmenüs werden erstellt:
  - **Menü** – enthält alle Steuerfunktionen
  - **About** – enthält Informationen zur Anwendung

Diese Struktur trennt Funktionssteuerung und Informationsbereich.

## 8.3 Menüeinträge und ihre Funktion

Die einzelnen Menüeinträge entsprechen den Hauptfunktionen der Anwendung:

- **Neu** – öffnet den Dialog zur Zeiteingabe
- **Start** – startet oder pausiert den Countdown
- **Reset** – setzt den Timer zurück
- **Exit** – beendet das Programm
- **Info** – zeigt Informationen zur Anwendung

Diese Einträge sind identisch zu den Buttons im Hauptfenster und bieten somit redundante Bedienmöglichkeiten.

## 8.4 Tastenkombinationen (Keyboard Shortcuts)

```

1 int ctrl = Toolkit.getDefaultToolkit().getMenuShortcutKeyMaskEx();

```

### Erklärung

- Ermittelt die systemabhängige Steuerungstaste
- Unter Windows: STRG
- Unter macOS: CMD
- sorgt für plattformunabhängige Tastenkombinationen

## 8.5 Zuweisung der Shortcuts

- Ctrl + N → Neuer Timer
- Ctrl + S → Start/Pause
- Ctrl + R → Reset
- Ctrl + X → Exit

```
1 newItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_N, ctrl));
2 startPauseItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_S, ctrl
  ));
3 resetItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_R, ctrl));
4 exitItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_X, ctrl));
```

Diese Tastenkombinationen ermöglichen eine schnelle Bedienung ohne Maus.

## 8.6 Bedeutung für die Benutzerfreundlichkeit

Das Menüsystem verbessert die Anwendung erheblich:

- alternative Bedienung neben Buttons
- schnellere Steuerung durch Tastenkürzel
- klare Strukturierung der Funktionen
- Standardverhalten moderner Desktop-Anwendungen

## 8.7 Zusammenfassung

Die Methode `createMenu()` implementiert eine vollständige Menüstruktur mit:

- Hauptmenü für Steuerfunktionen
- Informationsmenü (About)
- plattformunabhängigen Tastenkürzeln

Damit wird die Anwendung deutlich professioneller und benutzerfreundlicher gestaltet.

# 9 Klasse RoundButton

Die Klasse `RoundButton` erweitert die Standardklasse `JButton` und implementiert einen eigenen runden Button mit individuell angepasstem Aussehen und Verhalten.

Ziel dieser Klasse ist es, die Benutzeroberfläche moderner und intuitiver zu gestalten.

## 9.1 Vollständiger Quellcode

```
1 public class RoundButton extends JButton {
2
3     private Shape shape;
4
5     public RoundButton(String text) {
6         super(text);
7         setContentAreaFilled(false);
8         setFocusPainted(false);
9         setBorderPainted(false);
```

```

10     setOpaque(false);
11     setForeground(Color.WHITE);
12     setFont(new Font("SansSerif", Font.BOLD, 14));
13
14     addMouseListener(new MouseAdapter() {
15         @Override
16         public void mouseEntered(MouseEvent e) {
17             setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR)
18             );
19         }
20     });
21
22     @Override
23     protected void paintComponent(Graphics g) {
24         Graphics2D g2 = (Graphics2D) g.create();
25         g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
26             RenderingHints.VALUE_ANTIALIAS_ON);
27
28         if (getModel().isArmed()) {
29             g2.setColor(getBackground().darker());
30         } else if (getModel().isRollover()) {
31             g2.setColor(getBackground().brighter());
32         } else {
33             g2.setColor(getBackground());
34         }
35
36         g2.fillOval(0, 0, getWidth(), getHeight());
37         super.paintComponent(g);
38         g2.dispose();
39     }
40
41     @Override
42     public boolean contains(int x, int y) {
43         if (shape == null || !shape.getBounds().equals(getBounds())) {
44             shape = new Ellipse2D.Float(0, 0, getWidth(), getHeight());
45         }
46         return shape.contains(x, y);
47     }
48 }

```

## 9.2 Grundidee der Klasse

Die Klasse erweitert JButton, um einen runden Button zu erzeugen.

Standardmäßig sind Swing-Buttons rechteckig. Diese Klasse verändert das Rendering so, dass:

- der Button rund dargestellt wird
- Hover- und Klick-Effekte erhalten bleiben
- die Mausinteraktion korrekt funktioniert

## 9.3 Konstruktor und grundlegendes Design

Im Konstruktor wird das Standardverhalten des Buttons deaktiviert:

- setContentAreaFilled(false) → kein Standardhintergrund
- setBorderPainted(false) → kein Rahmen

- `setFocusPainted(false)` → kein Fokusrahmen
- `setOpaque(false)` → transparente Darstellung

Zusätzlich wird die Schrift und Farbe festgelegt, um ein einheitliches Design zu gewährleisten.

## 9.4 Mausverhalten

```
1 addMouseListener(new MouseAdapter() {
2     @Override
3     public void mouseEntered(MouseEvent e) {
4         setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
5     }
6 });
```

**Erklärung:**

- Beim Überfahren des Buttons mit der Maus ändert sich der Cursor
- Der Hand-Cursor signalisiert Interaktivität
- verbessert die Benutzerfreundlichkeit

## 9.5 Individuelles Rendering

```
1 protected void paintComponent(Graphics g)
```

Diese Methode überschreibt die Standarddarstellung des Buttons.

**Ablauf:**

- Umwandlung in `Graphics2D` für bessere Grafikfunktionen
- Aktivierung von Anti-Aliasing für glatte Kanten
- Auswahl der Farbe abhängig vom Zustand:
  - gedrückt → dunklere Farbe
  - Hover → hellere Farbe
  - Standard → normale Farbe
- Zeichnen eines gefüllten Kreises (`fillOval`)

Damit entsteht der runde Button-Effekt.

## 9.6 Klickbereich (Shape-Logik)

```
1 public boolean contains(int x, int y)
```

**Erklärung:**

- Standardmäßig wäre der klickbare Bereich rechteckig
- hier wird er auf eine Ellipse reduziert
- nur Klicks innerhalb des Kreises werden akzeptiert

Die `Shape`-Instanz wird nur neu berechnet, wenn sich die Größe des Buttons ändert.

## 9.7 Bedeutung für das Design

Diese Klasse verbessert die Anwendung deutlich:

- modernes, rundes UI-Design
- bessere visuelle Nutzerführung
- konsistente Interaktionseffekte
- erhöhte Benutzerfreundlichkeit

## 9.8 Zusammenfassung

Die Klasse `RoundButton` ersetzt `StandardButtons` durch eine eigene grafische Implementierung. Sie kombiniert:

- individuelles Rendering
- Mausinteraktion
- geometrische Klicklogik

Damit trägt sie wesentlich zum modernen Erscheinungsbild der Anwendung bei.

## 10 Klasse `TimeInputDialog`

Die Klasse `TimeInputDialog` erweitert `JDialog` und implementiert ein modales Eingabefenster zur Eingabe einer neuen Countdown-Zeit.

Sie dient als alternative Benutzeroberfläche zur Zeiteingabe und demonstriert den Einsatz von:

- inneren Klassen (Swing-Komponenten und Event-Listener)
- Lambda-Ausdrücken
- modularer GUI-Programmierung

### 10.1 Vollständiger Quellcode

```
1 public class TimeInputDialog extends JDialog {
2
3     private JTextField minutesField;
4     private JTextField secondsField;
5     private boolean confirmed = false;
6
7     public TimeInputDialog(JFrame parent) {
8         super(parent, "Neue Zeit", true);
9
10        setSize(300, 180);
11        setLocationRelativeTo(parent);
12        setLayout(new BorderLayout(10, 10));
13
14        // ICON oben links
15        JLabel icon = new JLabel("?");
16        icon.setOpaque(true);
17        icon.setBackground(new Color(46, 125, 50));
18        icon.setForeground(Color.BLACK);
19        icon.setFont(new Font("SansSerif", Font.BOLD, 20));
20        icon.setHorizontalAlignment(SwingConstants.CENTER);
```

```

21     icon.setPreferredSize(new Dimension(40, 40));
22
23     JPanel top = new JPanel(new FlowLayout(FlowLayout.LEFT));
24     top.add(icon);
25
26     add(top, BorderLayout.NORTH);
27
28     JPanel center = new JPanel(new FlowLayout());
29
30     minutesField = new JTextField(3);
31     secondsField = new JTextField(3);
32
33     center.add(new JLabel("Min:"));
34     center.add(minutesField);
35     center.add(new JLabel("Sek:"));
36     center.add(secondsField);
37
38     add(center, BorderLayout.CENTER);
39
40     JPanel bottom = new JPanel();
41
42     JButton ok = new JButton("OK");
43     JButton cancel = new JButton("Abbrechen");
44
45     ok.addActionListener(e -> {
46         confirmed = true;
47         dispose();
48     });
49
50     cancel.addActionListener(e -> dispose());
51
52     bottom.add(ok);
53     bottom.add(cancel);
54
55     add(bottom, BorderLayout.SOUTH);
56 }
57
58 public boolean isConfirmed() {
59     return confirmed;
60 }
61
62 public int getMinutes() {
63     return Integer.parseInt(minutesField.getText().trim());
64 }
65
66 public int getSeconds() {
67     return Integer.parseInt(secondsField.getText().trim());
68 }
69 }

```

## 10.2 Grundidee der Klasse

Die Klasse implementiert ein modales Eingabefenster, das die Hauptanwendung blockiert, bis der Benutzer eine Entscheidung trifft.

Ziel ist eine strukturierte und benutzerfreundliche Zeiteingabe.

### 10.3 Verwendung von inneren Klassen (Swing-Komponenten)

Die GUI besteht aus mehreren verschachtelten Komponenten, die typische **anonyme innere Klassen** in Swing darstellen:

- JLabel für das Icon
- JPanel für Layoutstruktur
- JButton für Benutzeraktionen

Diese Komponenten werden direkt innerhalb des Konstruktors erzeugt und verwendet, ohne eigene Klassen zu definieren.

Dies ist typisch für Swing-Programme, da UI-Elemente oft lokal und kontextgebunden sind.

### 10.4 Lambda-Ausdrücke für Event Handling

Ein zentraler Bestandteil dieser Klasse sind Lambda-Ausdrücke:

```
1 ok.addActionListener(e -> {  
2     confirmed = true;  
3     dispose();  
4 });  
5  
6 cancel.addActionListener(e -> dispose());
```

#### 10.4.1 Erklärung

Lambda-Ausdrücke ersetzen anonyme innere Klassen und machen den Code kompakter.

- `e -> { ... }` ersetzt eine vollständige `ActionListener`-Klasse
- reduziert Boilerplate-Code
- verbessert Lesbarkeit

#### 10.4.2 Vergleich zu klassischer Schreibweise

Ohne Lambda würde der Code so aussehen:

```
1 ok.addActionListener(new ActionListener() {  
2     @Override  
3     public void actionPerformed(ActionEvent e) {  
4         confirmed = true;  
5         dispose();  
6     }  
7 });
```

Die Lambda-Version ist deutlich kürzer und moderner.

### 10.5 Modale Fensterlogik

```
1 super(parent, "Neue Zeit", true);
```

**Erklärung:**

- `true` aktiviert Modalität
- Benutzer muss Dialog schließen, bevor Hauptfenster bedienbar ist

Dies verhindert inkonsistente Zustände während der Eingabe.



## 10.6 Datenzugriff über Getter-Methoden

Die Klasse stellt zwei Getter bereit:

- `getMinutes()` – liefert Minutenwert
- `getSeconds()` – liefert Sekundenwert

```
1 return Integer.parseInt(minutesField.getText().trim());
```

**Erklärung:**

- Zugriff erfolgt erst nach Dialogabschluss
- Daten werden dynamisch aus GUI gelesen

## 10.7 Bedeutung für die Softwarearchitektur

Diese Klasse verbessert die Architektur durch:

- klare Trennung von Eingabe und Hauptlogik
- Wiederverwendbarkeit
- bessere Benutzerführung
- moderne Eventbehandlung durch Lambdas

## 10.8 Zusammenfassung

Die Klasse `TimeInputDialog` zeigt moderne GUI-Programmierung in Java:

- Einsatz von inneren Klassen für UI-Komponenten
- Nutzung von Lambda-Ausdrücken für Events
- modulare und wiederverwendbare Struktur
- klare Trennung von Logik und Darstellung

# 11 Startpunkt der Anwendung: `main()`-Methode

Die `main()`-Methode ist der Einstiegspunkt des Programms. Sie initialisiert die grafische Benutzeroberfläche und stellt sicher, dass diese korrekt im Event Dispatch Thread (EDT) von Swing ausgeführt wird.

## 11.1 Vollständiger Quellcode

```
1 public static void main(String[] args) {  
2     SwingUtilities.invokeLater(() -> {  
3         CountdownWindowBuilder f = new CountdownWindowBuilder();  
4         f.setVisible(true);  
5     });  
6 }
```

## 11.2 Aufgabe der main-Methode

Die Hauptaufgaben dieser Methode sind:

- Starten der Anwendung
- Erzeugen des Hauptfensters
- Sicherstellen der korrekten GUI-Ausführung

Sie enthält bewusst keine Geschäftslogik, sondern nur die Initialisierung der Oberfläche.

## 11.3 Swing Event Dispatch Thread (EDT)

```
1 SwingUtilities.invokeLater(...)
```

**Erklärung:**

- Swing verwendet einen speziellen Thread für GUI-Operationen
- dieser Thread heißt **Event Dispatch Thread (EDT)**
- alle GUI-Zugriffe müssen im EDT erfolgen

Die Methode `invokeLater()` stellt sicher, dass der Code im richtigen Thread ausgeführt wird.

## 11.4 Verwendung eines Lambda-Ausdrucks

```
1 () -> {  
2     CountdownWindowBuilder f = new CountdownWindowBuilder();  
3     f.setVisible(true);  
4 }
```

**Erklärung:**

Der Lambda-Ausdruck ersetzt eine anonyme Runnable-Klasse:

- kompaktere Schreibweise
- moderne Java-Entwicklung
- gleiche Funktion wie klassische Runnable-Implementierung

## 11.5 Erzeugung der GUI

```
1 CountdownWindowBuilder f = new CountdownWindowBuilder();  
2 f.setVisible(true);
```

**Erklärung:**

- das Hauptfenster wird als Objekt erzeugt
- anschließend sichtbar gemacht

Ab diesem Moment übernimmt die Swing-Eventschleife die Kontrolle über das Programm.

## 11.6 Trennung von Logik und Darstellung

Ein wichtiges Architekturprinzip dieser Anwendung ist die klare Trennung:

- **main()-Methode:** nur Start und Initialisierung
- **CountdownWindowBuilder:** gesamte GUI- und Logikimplementierung

Diese Trennung hat mehrere Vorteile:

- bessere Wartbarkeit
- klare Struktur des Programms
- einfache Erweiterbarkeit
- saubere Verantwortlichkeiten (Single Responsibility Principle)

## 11.7 Zusammenfassung

Die **main()**-Methode dient ausschließlich als Startpunkt der Anwendung.

Sie stellt sicher, dass:

- die GUI im richtigen Thread gestartet wird
- keine Logik in den Einstiegspunkt ausgelagert wird
- die Architektur sauber und modular bleibt

Damit erfüllt sie eine zentrale Rolle in der Struktur des Programms, obwohl sie selbst nur wenige Zeilen Code enthält.

## 12 Mögliche Erweiterungen der Anwendung

Die entwickelte Countdown-Anwendung ist modular aufgebaut und kann in mehreren Bereichen sinnvoll erweitert werden. Durch die klare Trennung von Logik, Darstellung und Benutzerinteraktion ist eine Erweiterung ohne grundlegende Änderungen der Architektur möglich.

### 12.1 Erweiterung: Mehrere Timer gleichzeitig

Aktuell unterstützt die Anwendung nur einen aktiven Countdown.

Eine mögliche Erweiterung wäre:

- Verwaltung mehrerer paralleler Timer
- Anzeige in einer Liste oder Tabelle
- individuelle Start- und Stoppsteuerung pro Timer

Dies würde die Anwendung in Richtung eines Multi-Timer-Systems erweitern.

### 12.2 Erweiterung: Speicherung von Einstellungen

Derzeit gehen alle Einstellungen beim Schließen der Anwendung verloren.

Mögliche Verbesserung:

- Speichern der letzten Timer-Einstellungen in einer Datei
- Laden der gespeicherten Werte beim Programmstart
- Nutzung von JSON, Properties oder Serialisierung

Damit würde die Anwendung benutzerfreundlicher und nachhaltiger werden.

### 12.3 Erweiterung: Alarmfunktionen

Aktuell erfolgt nur ein einfacher Systemton am Ende.

Mögliche Erweiterungen:

- Auswahl verschiedener Alarmtöne
- visuelle Animationen beim Ablaufende
- Kombination aus Ton und Bildschirm-Effekten

Dies würde die Benutzerwahrnehmung verbessern.

### 12.4 Erweiterung: Benutzerdefinierte Oberfläche

Die Oberfläche könnte weiter angepasst werden:

- Dark-/Light-Mode Umschaltung
- frei wählbare Farben für Timer und Buttons
- Skalierung der Schriftgröße

Damit würde die Anwendung barrierefreier und flexibler werden.

### 12.5 Erweiterung: Verbesserung der Zeiteingabe

Die aktuelle Eingabe könnte erweitert werden durch:

- Slider statt Texteingabe
- direkte Eingabe in Sekunden
- Preset-Buttons (z.B. 1 min, 5 min, 10 min)

Dies würde die Bedienung weiter vereinfachen und Fehler reduzieren.

### 12.6 Erweiterung: Mobile oder Web-Version

Langfristig könnte die Anwendung portiert werden:

- Umsetzung als Web-App (z.B. mit JavaScript)
- mobile Version für Smartphones
- Synchronisation zwischen Geräten

Dies würde die Reichweite der Anwendung deutlich erhöhen.

### 12.7 Zusammenfassung

Die Anwendung ist bewusst einfach gehalten, bietet jedoch viele Erweiterungsmöglichkeiten.

Durch die modulare Struktur können neue Funktionen ergänzt werden, ohne die bestehende Architektur grundlegend zu verändern.

Dies macht das Projekt flexibel, skalierbar und gut wartbar.

## 13 Fazit und Zusammenfassung

Im Rahmen dieses Projekts wurde eine grafische Countdown-Anwendung in Java entwickelt, die auf der Swing-Bibliothek basiert. Ziel war es, einen funktionalen, benutzerfreundlichen und modular aufgebauten Timer zu erstellen.

### 13.1 Erreichte Ziele

Alle grundlegenden Anforderungen wurden erfolgreich umgesetzt:

- grafische Benutzeroberfläche mit Swing
- Countdown-Funktion mit Start, Pause und Reset
- Eingabe einer individuellen Zeit
- visuelle Rückmeldung durch Farben und Fortschrittsbalken
- akustisches Signal am Ende des Countdowns
- Menüstruktur mit Tastenkürzeln

Die Anwendung ist vollständig funktionsfähig und stabil im Gebrauch.

### 13.2 Technische Umsetzung

Während der Entwicklung wurden verschiedene zentrale Konzepte der Java-Programmierung angewendet:

- objektorientierte Programmierung (OOP)
- Ereignisgesteuerte Programmierung (Event Handling)
- Nutzung von Swing-Komponenten
- Lambda-Ausdrücke zur Vereinfachung von Event-Logik
- Timer-basierte Ablaufsteuerung

Besonders wichtig war die Trennung von Logik, Benutzeroberfläche und Steuerung, wodurch der Code übersichtlich und erweiterbar bleibt.

### 13.3 Stärken der Anwendung

Die entwickelte Anwendung zeichnet sich durch folgende Stärken aus:

- intuitive und moderne Benutzeroberfläche
- klare Struktur des Codes
- gute Erweiterbarkeit durch modulare Architektur
- visuelle und akustische Rückmeldungen
- einfache Bedienung durch Buttons und Tastenkürzel

### 13.4 Herausforderungen während der Entwicklung

Im Verlauf der Entwicklung traten mehrere Herausforderungen auf:

- Synchronisation von Timer und Benutzeroberfläche
- korrekte Umsetzung von Pause- und Fortsetz-Funktion
- Gestaltung einer benutzerfreundlichen Eingabeoberfläche
- Umsetzung dynamischer UI-Effekte (Farben, Blinken)

Diese Probleme konnten durch strukturierte Programmierung und iterative Verbesserung gelöst werden.

### **13.5 Ausblick**

Die Anwendung bietet zahlreiche Erweiterungsmöglichkeiten, wie z. B. mehrere Timer, Speicherung von Einstellungen oder eine erweiterte visuelle Gestaltung.

Dadurch eignet sich das Projekt gut als Grundlage für weiterführende Entwicklungen im Bereich GUI-Programmierung.

### **13.6 Schlusswort**

Zusammenfassend stellt das Projekt eine vollständige und funktionale Countdown-Anwendung dar, die grundlegende Konzepte der Java-Programmierung praxisnah kombiniert und erfolgreich umsetzt.

## A Klasse CountdownWindowBuilder

```

1 package Timer;
2
3 import javax.swing.*;
4 import javax.swing.border.EmptyBorder;
5 import java.awt.*;
6 import java.awt.event.KeyEvent;
7
8 public class CountdownWindowBuilder extends JFrame {
9
10     private JPanel contentPane;
11     private JLabel titleLabel;
12     private JLabel timeLabel;
13
14     private JButton startPauseButton;
15     private JButton resetButton;
16     private JButton newButton;
17
18     private JProgressBar progressBar;
19
20     private Timer timer;
21     private Timer pulseTimer;
22
23     private int initialSeconds = 0;
24     private int remainingSeconds = 0;
25     private boolean running = false;
26
27     private JMenuItem newItem, startPauseItem, resetItem, exitItem,
        aboutItem;
28
29     private final Color bg = new Color(24, 24, 28);
30     private final Color panelBg = new Color(32, 32, 38);
31     private final Color text = new Color(235, 235, 235);
32     private final Color accent = new Color(70, 130, 255);
33     private final Color green = new Color(46, 125, 50);
34     private final Color red = new Color(198, 40, 40);
35
36     public static void main(String[] args) {
37         SwingUtilities.invokeLater(() -> {
38             CountdownWindowBuilder f = new CountdownWindowBuilder();
39             f.setVisible(true);
40         });
41     }
42
43     public CountdownWindowBuilder() {
44
45         setTitle("Countdown");
46         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
47         setSize(700, 500);
48         setLocationRelativeTo(null);
49         setResizable(false);
50
51         contentPane = new JPanel();
52         contentPane.setBorder(new EmptyBorder(20, 20, 20, 20));
53         contentPane.setBackground(bg);
54         contentPane.setLayout(new BorderLayout(15, 15));

```

```

55     setContentPane(contentPane);
56
57     createMenu();
58
59     // ===== CENTER =====
60     JPanel center = new JPanel(new BorderLayout(10, 10));
61     center.setOpaque(false);
62
63     titleLabel = new JLabel("Countdown", SwingConstants.CENTER);
64     titleLabel.setFont(new Font("SansSerif", Font.BOLD, 30));
65     titleLabel.setForeground(text);
66
67     timeLabel = new JLabel("00:00", SwingConstants.CENTER);
68     timeLabel.setFont(new Font("SansSerif", Font.BOLD, 64));
69     timeLabel.setForeground(accent);
70
71     progressBar = new JProgressBar(0, 100);
72     progressBar.setStringPainted(true);
73     progressBar.setValue(100);
74     progressBar.setString("100%");
75     progressBar.setForeground(accent);
76     progressBar.setBackground(panelBg);
77
78     center.add(titleLabel, BorderLayout.NORTH);
79     center.add(timeLabel, BorderLayout.CENTER);
80     center.add(progressBar, BorderLayout.SOUTH);
81
82     contentPane.add(center, BorderLayout.CENTER);
83
84     // ===== BUTTONS =====
85     JPanel buttons = new JPanel(new FlowLayout(FlowLayout.CENTER,
86         15, 5));
87     buttons.setOpaque(false);
88
89     newButton = new RoundButton("Neu");
90     startPauseButton = new RoundButton("Start");
91     resetButton = new RoundButton("Reset");
92
93     style(newButton, accent);
94     style(startPauseButton, green);
95     style(resetButton, red);
96
97     resetButton.setEnabled(false);
98
99     buttons.add(newButton);
100    buttons.add(startPauseButton);
101    buttons.add(resetButton);
102
103    contentPane.add(buttons, BorderLayout.SOUTH);
104
105    // ===== MAIN TIMER =====
106    timer = new Timer(1000, e -> {
107
108        remainingSeconds--;
109        updateDisplay();
110
111        if (remainingSeconds <= 0) {
112            timer.stop();
113            pulseTimer.stop();
114
115            running = false;

```



```

115         startPauseButton.setText("Start");
116
117         timeLabel.setVisible(true);
118         timeLabel.setText("Fertig!");
119         timeLabel.setForeground(Color.RED);
120
121         Toolkit.getDefaultToolkit().beep();
122     }
123 });
124
125 // ===== PULSE TIMER =====
126 pulseTimer = new Timer(300, e -> {
127
128     if (remainingSeconds <= 5 && remainingSeconds > 0) {
129         timeLabel.setVisible(!timeLabel.isVisible());
130     } else {
131         timeLabel.setVisible(true);
132     }
133 });
134
135 pulseTimer.start();
136
137 // ===== ACTIONS =====
138
139 newButton.addActionListener(e -> openDialog());
140
141 startPauseButton.addActionListener(e -> startPause());
142
143 resetButton.addActionListener(e -> reset());
144
145 newItem.addActionListener(e -> openDialog());
146 startPauseItem.addActionListener(e -> startPauseButton.doClick()
147 );
148 resetItem.addActionListener(e -> reset());
149 exitItem.addActionListener(e -> System.exit(0));
150
151 aboutItem.addActionListener(e -> JOptionPane.showMessageDialog(
152     this,
153     "Countdown Timer",
154     "About",
155     JOptionPane.INFORMATION_MESSAGE
156 ));
157
158 // ===== START / PAUSE =====
159 private void startPause() {
160
161     if (!running) {
162
163         if (remainingSeconds <= 0) return;
164
165         timer.start();
166         pulseTimer.start();
167
168         running = true;
169         startPauseButton.setText("Pause");
170         resetButton.setEnabled(true);
171
172     } else {
173         timer.stop();
174         pulseTimer.stop();

```

```

175         running = false;
176         startPauseButton.setText("Fortsetzen");
177
178         timeLabel.setVisible(true);
179     }
180 }
181
182 // ===== RESET =====
183 private void reset() {
184
185     timer.stop();
186     pulseTimer.stop();
187
188     running = false;
189     remainingSeconds = initialSeconds;
190
191     updateDisplay();
192
193     timeLabel.setVisible(true);
194     timeLabel.setForeground(accent);
195
196     startPauseButton.setText("Start");
197     resetButton.setEnabled(false);
198 }
199
200 // ===== POPUP =====
201 private void openDialog() {
202
203     JDialog d = new JDialog(this, "Neue Zeit", true);
204     d.setSize(280, 160);
205     d.setLocationRelativeTo(this);
206     d.setLayout(new FlowLayout());
207
208     JTextField m = new JTextField(String.valueOf(initialSeconds /
209         60), 3);
210     JTextField s = new JTextField(String.valueOf(initialSeconds %
211         60), 3);
212
213     JButton ok = new JButton("OK");
214
215     ok.addActionListener(ev -> {
216         try {
217             int mm = Integer.parseInt(m.getText().trim());
218             int ss = Integer.parseInt(s.getText().trim());
219
220             if (mm < 0 || ss < 0 || ss > 59) {
221                 JOptionPane.showMessageDialog(d,
222                     "Bitte gueltige Zeit eingeben:\nMin >= 0\
223                     nSek 0-59",
224                     "Ungueltig",
225                     JOptionPane.WARNING_MESSAGE);
226                 return;
227             }
228
229             timer.stop();
230             pulseTimer.stop();
231
232             running = false;

```

```

233         startPauseButton.setText("Start");
234
235         setTime(mm, ss);
236
237         resetButton.setEnabled(true);
238
239         d.dispose();
240
241     } catch (NumberFormatException ex) {
242         JOptionPane.showMessageDialog(d,
243             "Bitte nur Zahlen eingeben!",
244             "Fehler",
245             JOptionPane.ERROR_MESSAGE);
246     }
247 }));
248
249 d.add(new JLabel("Min:"));
250 d.add(m);
251 d.add(new JLabel("Sek:"));
252 d.add(s);
253 d.add(ok);
254
255 d.setVisible(true);
256 }
257
258 // ===== TIME =====
259 private void setTime(int m, int s) {
260
261     initialSeconds = m * 60 + s;
262     remainingSeconds = initialSeconds;
263
264     updateDisplay();
265 }
266
267 // ===== DISPLAY =====
268 private void updateDisplay() {
269
270     int min = remainingSeconds / 60;
271     int sec = remainingSeconds % 60;
272
273     timeLabel.setText(String.format("%02d:%02d", min, sec));
274
275     // Farbe
276     if (remainingSeconds <= 5 && remainingSeconds > 0) {
277         timeLabel.setForeground(Color.RED);
278     } else if (remainingSeconds <= 10) {
279         timeLabel.setForeground(Color.RED.darker());
280     } else {
281         timeLabel.setForeground(accent);
282     }
283
284     if (initialSeconds > 0) {
285         int percent = (int) (remainingSeconds * 100.0 /
286             initialSeconds);
287         progressBar.setValue(percent);
288         progressBar.setString(percent + "%");
289     }
290 }
291
292 // ===== STYLE =====
293 private void style(RoundButton b, Color c) {

```

```

293         b.setBackground(c);
294         b.setForeground(Color.WHITE);
295         b.setPreferredSize(new Dimension(90, 90));
296     }
297
298     // ===== MENU =====
299     private void createMenu() {
300
301         JMenuBar bar = new JMenuBar();
302         setJMenuBar(bar);
303
304         JMenu m1 = new JMenu("Menue");
305         JMenu m2 = new JMenu("About");
306
307         bar.add(m1);
308         bar.add(m2);
309
310         newItem = new JMenuItem("Neu");
311         startPauseItem = new JMenuItem("Start");
312         resetItem = new JMenuItem("Reset");
313         exitItem = new JMenuItem("Exit");
314         aboutItem = new JMenuItem("Info");
315
316         m1.add(newItem);
317         m1.add(startPauseItem);
318         m1.add(resetItem);
319         m1.add(exitItem);
320
321         m2.add(aboutItem);
322
323         int ctrl = Toolkit.getDefaultToolkit().getMenuShortcutKeyMaskEx
324             ();
325
326         newItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_N,
327             ctrl));
328         startPauseItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_S, ctrl));
329         resetItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_R, ctrl));
330         exitItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_X, ctrl));
331     }
332 }

```

## B Klasse RoundButton

```

1 package Timer;
2
3 import javax.swing.*;
4 import java.awt.*;
5 import java.awt.event.MouseAdapter;
6 import java.awt.event.MouseEvent;
7 import java.awt.geom.Ellipse2D;
8
9 public class RoundButton extends JButton{
10
11     private Shape shape;
12
13     public RoundButton(String text) {

```

```

14     super(text);
15     setContentAreaFilled(false);
16     setFocusPainted(false);
17     setBorderPainted(false);
18     setOpaque(false);
19     setForeground(Color.WHITE);
20     setFont(new Font("SansSerif", Font.BOLD, 14));
21
22     addMouseListener(new MouseAdapter() {
23         @Override
24         public void mouseEntered(MouseEvent e) {
25             setCursor(Cursor.getPredefinedCursor(Cursor.HAND_CURSOR));
26         }
27     });
28 }
29
30 @Override
31 protected void paintComponent(Graphics g) {
32     Graphics2D g2 = (Graphics2D) g.create();
33     g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.
        VALUE_ANTIALIAS_ON);
34
35     if (getModel().isArmed()) {
36         g2.setColor(getBackground().darker());
37     } else if (getModel().isRollover()) {
38         g2.setColor(getBackground().brighter());
39     } else {
40         g2.setColor(getBackground());
41     }
42
43     g2.fillOval(0, 0, getWidth(), getHeight());
44     super.paintComponent(g);
45     g2.dispose();
46 }
47
48 @Override
49 public boolean contains(int x, int y) {
50     if (shape == null || !shape.getBounds().equals(getBounds())) {
51         shape = new Ellipse2D.Float(0, 0, getWidth(), getHeight());
52     }
53     return shape.contains(x, y);
54 }
55 }

```

## C Klasse TimeInputDialog

```

1 package Timer;
2
3 import javax.swing.*;
4 import java.awt.*;
5
6 public class TimeInputDialog extends JDialog {
7
8     private JTextField minutesField;
9     private JTextField secondsField;
10    private boolean confirmed = false;
11
12    public TimeInputDialog(JFrame parent) {
13        super(parent, "Neue Zeit", true);

```

```

14
15     setSize(300, 180);
16     setLocationRelativeTo(parent);
17     setLayout(new BorderLayout(10, 10));
18
19     // ICON oben links
20     JLabel icon = new JLabel("?");
21     icon.setOpaque(true);
22     icon.setBackground(new Color(46, 125, 50)); // gruen
23     icon.setForeground(Color.BLACK);
24     icon.setFont(new Font("SansSerif", Font.BOLD, 20));
25     icon.setHorizontalAlignment(SwingConstants.CENTER);
26     icon.setPreferredSize(new Dimension(40, 40));
27
28     JPanel top = new JPanel(new FlowLayout(FlowLayout.LEFT));
29     top.add(icon);
30
31     add(top, BorderLayout.NORTH);
32
33     JPanel center = new JPanel(new FlowLayout());
34
35     minutesField = new JTextField(3);
36     secondsField = new JTextField(3);
37
38     center.add(new JLabel("Min:"));
39     center.add(minutesField);
40     center.add(new JLabel("Sek:"));
41     center.add(secondsField);
42
43     add(center, BorderLayout.CENTER);
44
45     JPanel bottom = new JPanel();
46
47     JButton ok = new JButton("OK");
48     JButton cancel = new JButton("Abbrechen");
49
50     ok.addActionListener(e -> {
51         confirmed = true;
52         dispose();
53     });
54
55     cancel.addActionListener(e -> dispose());
56
57     bottom.add(ok);
58     bottom.add(cancel);
59
60     add(bottom, BorderLayout.SOUTH);
61 }
62
63 public boolean isConfirmed() {
64     return confirmed;
65 }
66
67 public int getMinutes() {
68     return Integer.parseInt(minutesField.getText().trim());
69 }
70
71 public int getSeconds() {
72     return Integer.parseInt(secondsField.getText().trim());
73 }
74 }

```

---